RISC-V Ottawa

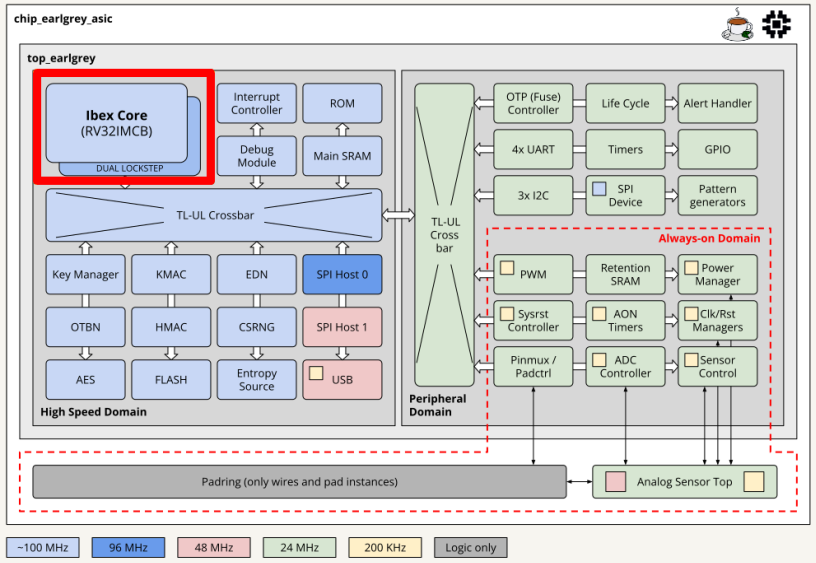
A side quest into the FOSSi universe

A RISC-V CPU on a $20 FPGA using zero proprietary tools

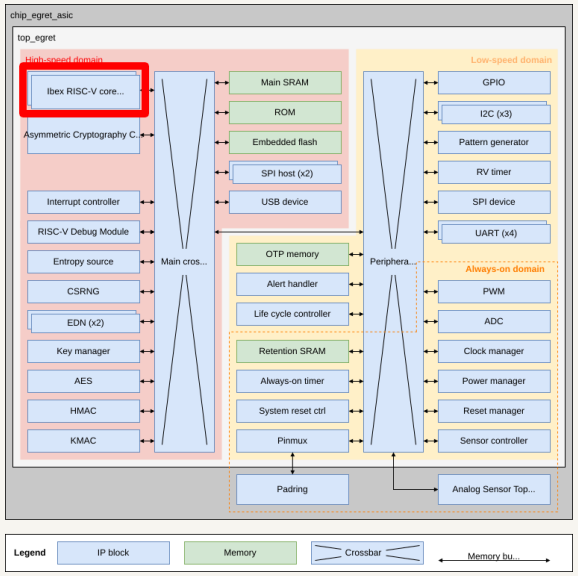
July 2026 · Yusef Karim

No money? No problem.

Last month I pitched the **hardware root of trust** (HWRoT) project, and asked if anyone had an **$8k–16k** FPGA they could lend...**nobody did** (rude).



OpenTitan Earl Grey



Pavona Egret

Turns out the CPU used by both HWRoT reference implementations fits on a $20 FPGA!

The side quest

Get the **real RISC-V core** used by OpenTitan and Pavona running on the **$20 FPGA** I had lying around.

First, a confession

A month ago I had **never** built anything for an FPGA, and had **never touched** a single tool in this talk.

LiteX, Yosys, nextpnr, Apicula, openFPGALoader…all brand new to me.

What I had

An interest in RISC-V, a $20 board, and a stubborn desire for hands-on experience without waiting for a $16k board.

What I did *not* have

Any idea what a **slang frontend**, a **distributed-RAM regfile**, or the **memory layout** of the Ibex CPU was.

This is a beginner's field report. Expect rabbit holes.

Latch-Up 2026 - a motivational drug

New to hardware? FOSSi to the rescue!

Free and Open Source Silicon (FOSSi): making the entire stack open.

The ISA

RISC-V is an open instruction set anyone can implement.

The RTL

Ibex and cores like it are real, production CPUs you can read, change, and build.

The tools ⭐

The **FOSSi CAD tools** make synthesis, place & route, and flashing, all open too.

Open ISA, open implementations, open tools

Don't forget about the final boss: the PDK. Even the fab recipe can be open: PDKs like **SkyWater 130** and **IHP 130**, plus shared tape-out programs (**TinyTapeout**, the **SSCS Chipathon**, academic MPW shuttles) that put real designs onto real silicon. *Not today's side quest, but the same idea one level deeper.*

What I built

A complete little RISC-V **system-on-chip**, generated by **LiteX**, running on a board that costs about as much as a few boxes of Timbits.

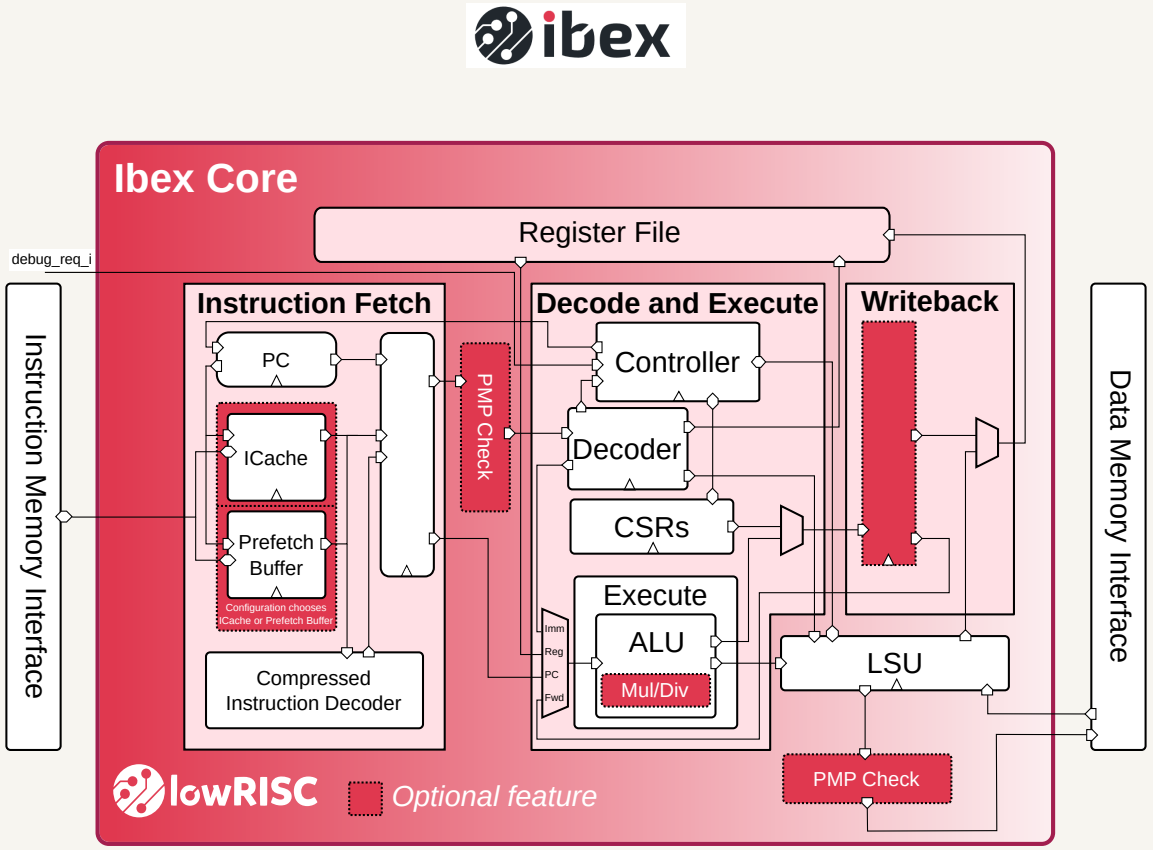
The core

lowRISC **Ibex**: an **RV32IMC**, machine-mode CPU - PLUS a **UART**, memory, and a **Wishbone** bus tying it all together.

The board · $20

Sipeed **Tang Nano 9K**: a Gowin **GW1NR-9** FPGA, with **8,640** LUT4s to squeeze into.

The Ibex core



The stock Ibex two-stage pipeline.

Ibex "small"

RV32IMC, machine-mode only. Two-stage pipeline, no cache, no branch predictor, no PMP.

The one override

`RegFile=1` – swap the flip-flop register file for a **distributed-RAM** one. Everything else is stock `ibex_top.sv`.

How it talks

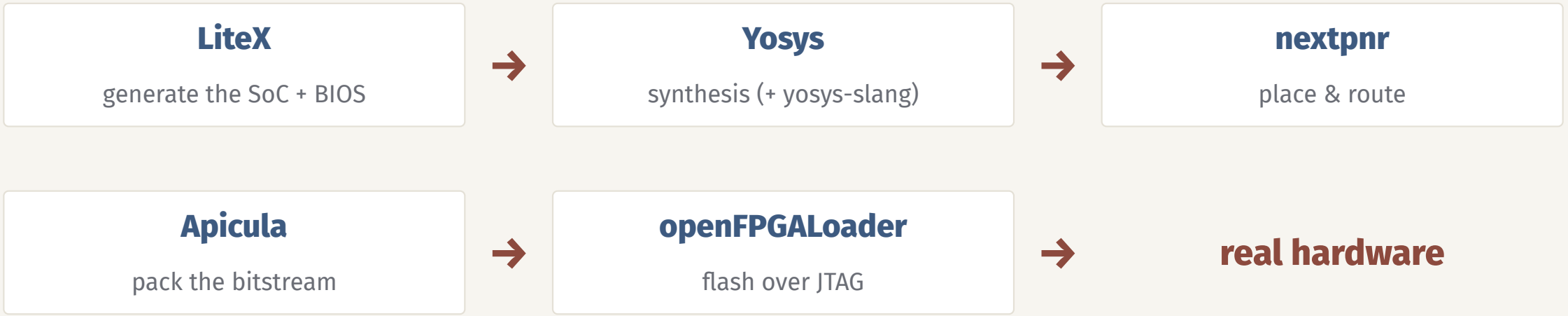
Ibex speaks **OBI** (shout out OpenHW Foundation); LiteX bridges each bus to **Wishbone**, tying core, RAM, and peripherals together.

What's on the bus

64 KB ROM, 8 KB SRAM, 8 KB main RAM, a **UART**, a **timer**, **GPIO** LEDs, and 4 MB of SPI flash. Two interrupts.

The open flow

Every box below is open source, and the Yosys team provides all of them in one download.



RISC-V GCC builds the firmware; **LiteX** wires it into a bootable BIOS. No vendor tools anywhere.

All of it from the OSS CAD Suite, wrapped in one container image.

One container, a handful of commands

The **OSS CAD Suite**, plus RISC-V GCC and LiteX, wrapped in one container. No local host installation, no vendor login.

`make image`	build the container: toolchain + LiteX
`make soc`	generate, synthesize, and place & route the SoC
`make flash`	load the bitstream and BIOS onto the board
`make run APP=blink_irq`	cross-compile an app and serial-boot it

From an empty checkout to a booting chip, easily and reproducibly.

It works


```
      __   _ __      _  __
     / /  (_) /____ | |/_/
    / /__/ / __/ -_)>  <
   /____/_/\__/\__/_/|_|
  Build your hardware, easily!

 (c) Copyright 2012-2026 Enjoy-Digital
 (c) Copyright 2007-2015 M-Labs

 BIOS built on Jul 14 2026 13:45:43
 BIOS CRC passed (4b008265)

 LiteX git sha1: 3b1fc6aa8

--=============== SoC ==================--
CPU:            Ibex @ 24MHz
BUS:            wishbone 32-bit data/32-bit addr
CSR:            32-bit data big ordering
ROM:            64.0KiB
SRAM:           8.0KiB
FLASH:          4.0MiB
MAIN RAM:       8.0KiB

--=========== Initialization ============--
Memtest at 0x40000000 (8.0KiB)...
  Write: 0x40000000-0x40002000 8.0KiB
   Read: 0x40000000-0x40002000 8.0KiB
Memtest OK
Memspeed at 0x40000000 (Sequential, 8.0KiB)...
  Write speed: 519.2KiB/s
   Read speed: 431.9KiB/s

Initializing w25q32 SPI Flash @0x00000000...
SPI Flash clk configured to 12 MHz (div: 2)
Memspeed at 0 (Sequential, 4.0KiB)...
   Read speed: 235.3KiB/s
Memspeed at 0 (Random, 4.0KiB)...
   Read speed: 30.0KiB/s

--=============== Boot ================--
Booting from serial...
Press Q or ESC to abort boot completely.
sL5DdSMmkekro
Timeout
Booting from flash...
Copying 0x00100000 to 0x40000000 (1696 bytes)...
[########################################]
Executing booted program at 0x40000000

--============== Liftoff! ==============--
```

It boots. On real hardware, over a serial console:

- `CPU: Ibex @ 24MHz`
- `BIOS CRC passed`
- `Memtest OK`
- a live `litex>` prompt that answers back

...then it runs my own bare-metal **`blink_irq`** app: a hardware timer fires a **periodic interrupt** to toggle an LED, with the same ISR also servicing the UART.

Fits in 77% of the LUT4s (6,669 / 8,640)

Demo

Let’s boot it.

`make serial`	tap reset ⟶ BIOS banner, then a live litex> prompt
`make flash-app APP=blink_irq`	write blink_irq into the board's SPI flash
`make reset`	reload from flash ⟶ a timer IRQ blinks the LED, survives a power cycle

New to hardware? Oh boy...

Every one of these was its own little rabbit hole. *It was ~~a headache~~ fun to figure out.*

Register file overflowed the chip at 106%

➜ a distributed-RAM regfile drops it to about 80%

On-board HyperRAM: 99.6% memtest corruption

➜ skip it; run from 8 KB of BRAM instead

At 27 MHz the CPU was dead silent

➜ ~~13.5 MHz works though...still not fully sure why.~~

Mouad solved this like a boy genius. He found a toolchain bug, **PR coming soon**!

115200 baud overran the receiver

➜ drop to 9600 and uploads work every time

My main contribution and RISC-V assembly

The code just **never booted**. The fix was simple, but realizing it...took awhile.

The wrong offset

Ibex's hardware reset vector is at `0x80`. The stock crt0 put `_start` at `0x00` and pushed the trap table up to `0x100`. On reset the core jumped to `0x80` and landed at the **wrong instructions**.

The missing .norvc

The vector table also lacked `.norvc`, so its `j` entries could compress to **2 bytes** – leaving the table **misaligned** under a vectored `mtvec`.

Fix: `.option norvc` forces **4-byte** entries, the 32-entry table now spans `0x00–0x7F`, and aligning `_start` right after the table places it right at `0x80`, exactly where Ibex resets.

Giving back (as a total beginner)

Two fixes landed **upstream in LiteX**, and the most useful one wasn't code I wrote.

Making Ibex build at all

I filed the issue: stock Yosys can't parse Ibex's SystemVerilog. A maintainer took my proposal and wrote a proper **slang** fix – but couldn't run it, **no board**.

I tested using my board, and provided two changes that helped finalize the patch.

Merged: PR #2510

The reset-vector fix

The crt0 bug from the last slide – tracked down, patched, and sent up. This one was mine, end to end.

Merged: PR #2487

"I have the weird board and I'll test everything" turns out to be a real contribution.

LiteX developers were very kind and fast to respond, 10/10 would contribute again.

Where to find it

If you have a Tang Nano, go try it out!

Repo github.com/riscv-ottawa/ibex-tang-nano-oss-cad

Try it Grab a $20 **Tang Nano 9K** and follow the README, top to bottom.

Next Point this open flow at something bigger...

(small components from a certain root of trust, maybe 👀).